L'analyse orientée objet

Cette fiche présente :

1 **Introduction et enjeux de l'analyse orientée objet**

2 **L'analyse du métier : modéliser l'organisation et les processus**

3 **L'analyse du système informatique : formaliser les services et la structure**

1. Introduction et enjeux de l'analyse orientée objet

1.1. L'analyse orientée objet : une phase clé du cycle de vie logiciel

L'**analyse orientée objet** s'inscrit dans le cadre plus large du cycle de vie d'un projet informatique, où elle joue un rôle charnière entre la compréhension du métier et la conception technique du système. Cette phase se décline en deux temps distincts, quel que soit l'approche choisie :

1. **L'analyse du métier** : étudier les activités, les circuits, les règles de gestion et les informations échangées dans le domaine étudié, afin de comprendre l'environnement et l'organisation du travail. Cette étape est cruciale pour modéliser l'existant et identifier les besoins métiers avant toute automatisation.
2. **L'analyse du système informatique** : recenser, décrire et formaliser les **services** que le futur logiciel devra offrir pour répondre aux besoins exprimés par la maîtrise d'ouvrage et les utilisateurs, tout en s'intégrant harmonieusement dans le système d'information existant.

Dans une approche orientée objet, ces deux phases sont souvent **imbriquées** : l'analyse du métier permet de définir le cadre organisationnel, tandis que l'analyse du système informatique se concentre sur la modélisation des **fonctionnalités** et des **interactions** nécessaires pour répondre aux besoins identifiés.

1.2. Spécificités de l'analyse orientée objet : focus sur les cas d'utilisation et l'architecture

L'analyse orientée objet se distingue des méthodes systémiques (comme Merise) par son **centrage sur les cas d'utilisation** et son **approche itérative et incrémentale**. Voici ses principes fondateurs :

- **Processus centré sur les cas d'utilisation** :
  Les cas d'utilisation sont **au cœur du projet**. Ils permettent de recueillir les besoins d'informatisation, d'identifier les services attendus du système, de valider l'avancement des travaux et de planifier les tests. Leur force réside dans leur **accessibilité** : rédigés dans un

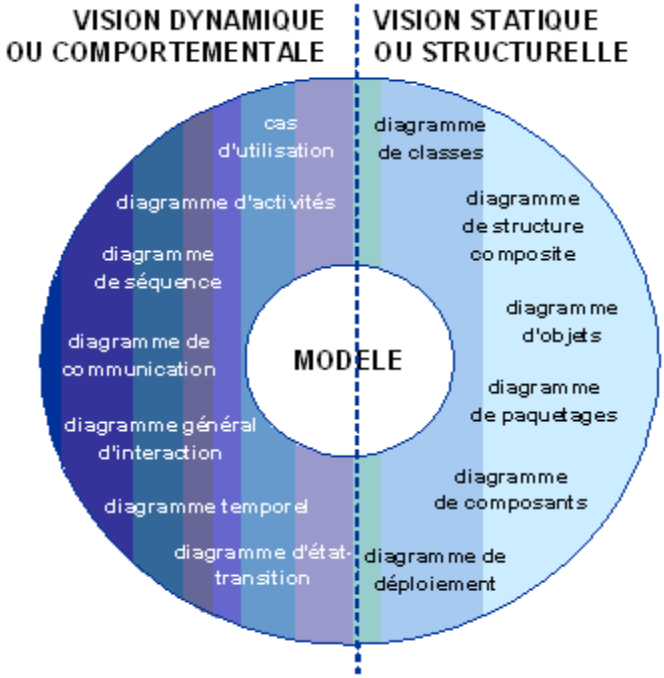
langage naturel, ils sont compréhensibles par tous les acteurs (MOA, MOE, utilisateurs), sans nécessiter de compétences techniques spécifiques.
Exemple : Un cas d'utilisation « Enregistrer une commande » décrit les interactions entre un acteur (le client ou le vendeur) et le système pour créer une commande, sans détailler comment cette opération sera implémentée techniquement.

- **Processus centré sur l'architecture** :
  L'architecture du système est un **guidage constant** tout au long du développement. En phase d'analyse, elle permet de :
  - **Décomposer les concepts** du domaine en objets et en classes, pour offrir une vision structurée et cohérente du futur système.
  - **Construire une architecture applicative** modulaire et évolutive, facilitant la réutilisabilité des composants et l'adaptabilité aux changements.
    Exemple : Une classe « Client » peut regrouper des attributs (nom, adresse, numéro de téléphone) et des opérations (créer, modifier, consulter), tandis qu'une classe « Commande » gère les informations liées aux achats (produits, date, statut).
- **Processus itératif et incrémental** :
  L'approche objet préconise une **construction progressive** du système, par l'ajout successif de modules indépendants (les « briques » du système). Chaque itération permet d'intégrer de nouveaux services, d'affiner les besoins et de valider les livrables (documents, prototypes, modules fonctionnels).
  Exemple : Un système de gestion de bibliothèque peut être développé par étapes : d'abord la gestion des livres, puis les emprunts, puis les retours, etc.

1.3. UML : un langage unifié pour l'analyse orientée objet

UML (Unified Modeling Language) est le **langage de modélisation** par excellence pour l'analyse orientée objet. Né de la volonté de **standardiser** les méthodes d'analyse et de conception objet, UML propose une **boîte à outils** de diagrammes permettant de représenter le système sous différents angles :

- **Vue statique** (structure) : diagrammes de classes, d'objets, de composants.
- **Vue dynamique** (comportement) : diagrammes de séquence, d'activités, de cas d'utilisation, d'états-transitions.



Origines d'UML :
UML est issu de la fusion de trois méthodes dominantes des années 1990 :

- **OOD** (Object Oriented Design, Grady Booch) : axée sur la conception orientée objet.
- **OMT** (Object Modeling Technique, James Rumbaugh) : adaptée aux systèmes manipulant de grandes quantités de données.
- **OOSE** (Object Oriented Software Engineering, Ivar Jacobson) : centrée sur les cas d'utilisation pour recueillir les besoins.

Évolutions majeures :

- **UML 2.x** a introduit des améliorations significatives :
  - Enrichissement des **diagrammes de séquence** (modélisation du temps réel, structures de contrôle comme les boucles ou alternatives).
  - Compléments aux **diagrammes d'activités** (actions, nœuds de contrôle, flots d'objets).
  - Meilleure modélisation de l'**architecture par composants** (ports, interfaces, assemblage).

Pourquoi UML est-il incontournable ?

- **Neutre** : applicable à tout type de projet (gestion, industriel, temps réel).
- **Flexible** : peut être utilisé avec n'importe quelle méthode objet (RUP, ITEOR/OO, Merise+UML).
- **Pédagogique** : ses diagrammes sont **visuels et compréhensibles** par tous les intervenants.

1.4. Les limites et défis de l'analyse orientée objet

Malgré ses avantages, l'analyse orientée objet présente des **défis** :

- **Complexité initiale** : les concepts (objets, classes, héritage) peuvent sembler abstraits pour les non-informaticiens.
- **Choix de la méthode** : l'absence de méthode standardisée peut conduire à des **démarches incohérentes** si chaque équipe utilise sa propre approche.
- **Focus sur le système informatique** : certaines méthodes orientées objet négligent l'analyse du métier, au risque de produire un système **déconnecté des réalités organisationnelles**.
- **Équilibre entre abstraction et détail** : il faut savoir **modéliser juste ce qu'il faut**, sans tomber dans le piège du « tout objet » ou de la sur-modélisation.

Synthèse :
L'analyse orientée objet repose sur trois piliers :

1. **Comprendre le métier** pour définir le cadre organisationnel.
2. **Formaliser les besoins** via les cas d'utilisation et les diagrammes UML.
3. **Construire une architecture modulaire** pour un système évolutif et réutilisable.

2. L'analyse du métier : modéliser l'organisation et les processus

2.1. Pourquoi analyser le métier ?

L'analyse du métier est **optionnelle mais cruciale** dans les cas suivants :

- **Nouveaux métiers** : lorsque le domaine étudié est inconnu ou en cours de définition.
- **Processus divergents** : si les pratiques actuelles sont incohérentes ou inefficaces.
- **Recours à un progiciel** : pour adapter les processus métiers aux fonctionnalités du logiciel choisi.

Dans ces situations, l'analyse du métier permet de :

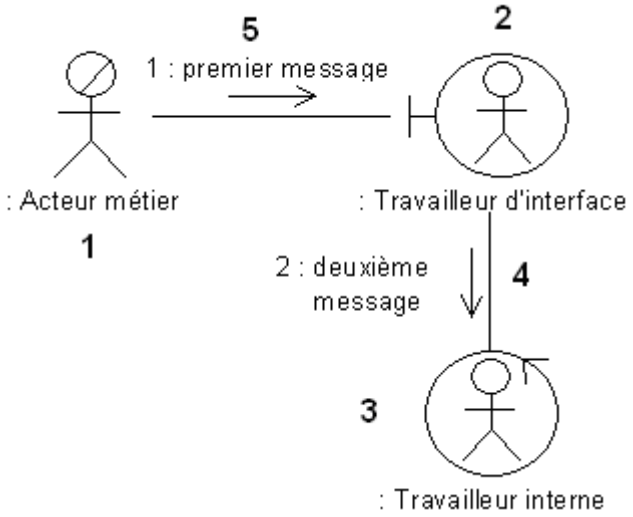
- **Formaliser l'organisation** des activités et des rôles des acteurs.
- **Identifier les règles de gestion** et les flux d'informations.
- **Préparer l'automatisation** en alignant les processus métiers sur les fonctionnalités du futur système.

2.2. Les diagrammes UML pour l'analyse du métier

Plusieurs diagrammes UML sont adaptés à cette phase, chacun répondant à un besoin spécifique :

2.2.1. Le diagramme de communication : les échanges entre acteurs

Cette démarche vise à représenter de manière exhaustive les échanges et dépendances entre les différents acteurs métiers (clients, fournisseurs, services internes) et le système d'information, afin d'en clarifier les flux fonctionnels et les responsabilités respectives.

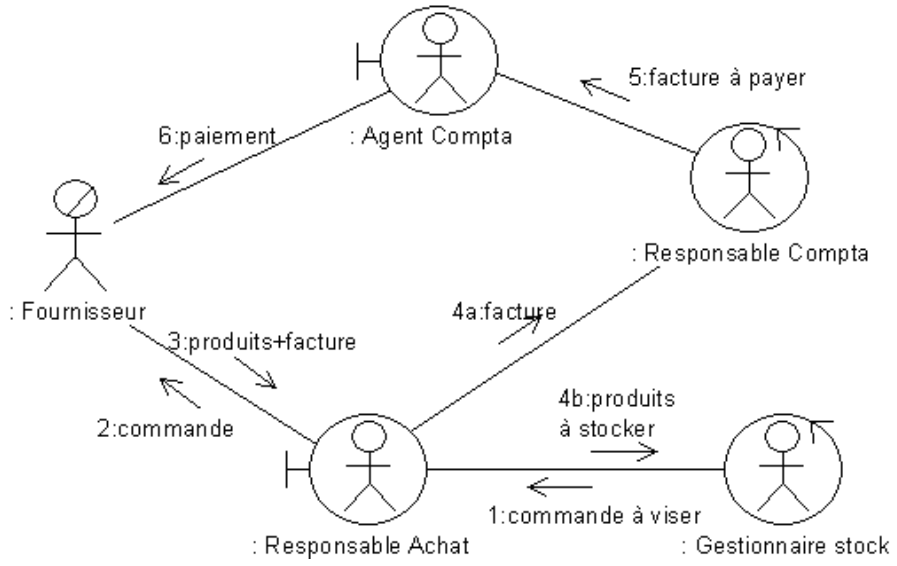


Concepts clés :

- **Acteur métier** : entité externe sollicitant le système (ex : un client).
- **Travailleur** : rôle interne participant à la production des services (ex : un employé du service client).
    - **Travailleur d'interface** : en contact avec les acteurs métiers (ex : un guichetier).
    - **Travailleur interne** : n'interagit qu'avec d'autres travailleurs (ex : un gestionnaire de stock).
- **Message** : échange d'informations entre deux rôles.

Exemple :

> *Le travailleur interne* **Gestionnaire stock** *transmet une commande à viser au travailleur d'interface* **Responsable Achat**, *qui l'envoie au* **Fournisseur** *après visa. Le Fournisseur envoie les produits et la facture au Responsable Achat, qui vérifie le contenu de la livraison et la facturation, avant d'envoyer la facture au travailleur interne* **Responsable Compta** *et les produits au Gestionnaire stock.*



2.2.2. Le diagramme d'activités métier : l'organisation des tâches

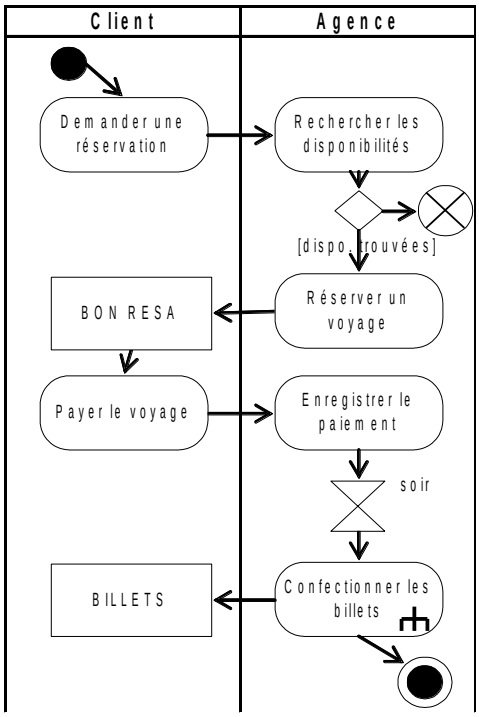
Cette approche a pour but de cartographier de manière détaillée la séquence des opérations métiers, en précisant pour chaque étape les acteurs responsables, leurs interactions et les décisions associées, afin d'assurer une compréhension globale et cohérente des processus.

Concepts clés :

- **Action** : unité de travail (ex : « Vérifier la présence des pièces »).
- **Activité** : ensemble d'actions liées (ex : « Traiter un dossier »).
- **Couloir (swimlane)** : colonne dédiée à un acteur, regroupant ses tâches.
- **Nœuds de contrôle** : points de décision, fourchettes (parallélisme), jointures (synchronisation).
- **Flots de contrôle** : enchaînement des actions.
- **Flots d'objets** : transmission d'informations entre actions.

Exemple :

> *Un client effectue une* **demande de réservation**. *Si l'agence trouve des* **disponibilités**, *elle réalise une* **réservation de voyage**. *Un* **bon de réservation** *est remis au client. Lorsqu'il* **paie le voyage**, *l'agence* **enregistre le paiement**.



Notation UML :

- **Couloir** : colonne avec un nom (ex : « Client », « Agence »).
- **Action** : rectangle aux coins arrondis (ex : « Réserver un voyage »).
- **Nœud initial** : cercle plein.
- **Nœud final** : cercle plein entouré d'un cercle.
- **Point de décision** : losange vide.
- **Flot de contrôle** : flèche orientée.

Démarche :

1. **Identifier les acteurs** (clients, employés, services).
2. **Lister les tâches** pour chaque acteur, en partant d'une sollicitation (ex : une demande client) jusqu'à la production d'un résultat.
3. **Ajouter les flots de contrôle** pour représenter l'ordre des actions et les points de décision.
4. **Détailler les cas exceptionnels** (ex : dossier incomplet, produit indisponible).

3. L’analyse du système informatique : formaliser les services et la structure

3.1. Objectifs de l’analyse du système informatique

Cette phase vise à **traduire les besoins métiers en spécifications fonctionnelles** pour le futur logiciel, en se concentrant sur :

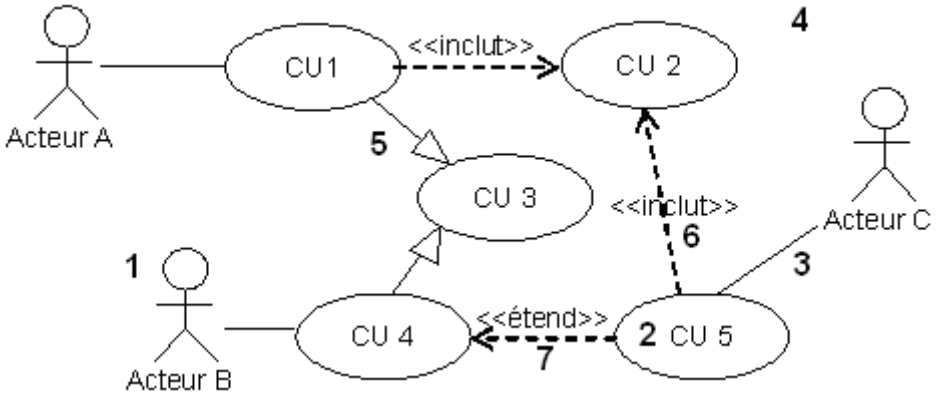
- **Les services offerts** par le système (fonctionnalités).
- **Les données gérées** (classes et attributs).
- **Les interactions** entre le système et ses utilisateurs (cas d’utilisation).

L’analyse du système informatique est **au cœur de la proposition d’informatisation** et doit répondre à la question : **« Que doit faire le système ? »**, et non **« Comment le fera-t-il ? »** (cette question relève de la conception technique).

3.2. Les diagrammes UML pour l’analyse du système informatique

3.2.1. Le diagramme de cas d’utilisation : les fonctionnalités du système

Cette étape consiste à **lister systématiquement l’ensemble des fonctionnalités** attendues du projet, tout en **associant à chacune les profils d’acteurs** (rôles ou services) habilités à les exploiter.



Concepts clés :

- **Système informatique** : l’application en cours de construction.
- **Acteur** : utilisateur (personne ou autre système) en interaction avec le système.
- **Cas d’utilisation** : service offert par le système (ex : « Enregistrer une commande »).
- **Scénario** :
    - **Flot normal** : chemin principal pour rendre le service.
    - **Flots alternatifs** : variantes utiles du flot normal.
    - **Flots exceptionnels** : cas ne permettant pas d’aboutir au résultat attendu.

Exemple :

Acteur : **Employé de vente** → *Cas d'utilisation : « Enregistrer une commande » → Scénario : l'employé saisit les produits commandés et valide. Si le client n'existe pas, un flot alternatif « Client inexistant » est déclenché.*

NOM DU CAS D'UTILISATION

Acteurs :

Événement déclencheur :

Événement généré :

Nature du cas d'utilisation :

Périodicité d'utilisation :

Flot normal :

1- L'acteur choisit…

2- Le système affiche…

3- L'acteur sélectionne…

4- Le système enregistre…

Flots alternatifs

2a- Si l'acteur a choisi…, alors le système affiche…

4a- Si tel cas, alors le système ne peut enregistrer et affiche…

Notation UML :

- **Acteur** : Humain.
- **Cas d'utilisation** : ellipse.
- **Association** : ligne entre acteur et cas d'utilisation.
- **Généralisation** : flèche creuse (ex : « S'authentifier » → « S'authentifier par badge »).
- **Inclusion/Extension** : flèches en pointillés (pour les services communs ou optionnels).

Démarche :

1. **Identifier les services** offerts par le système (à partir des cas d'utilisation du métier ou des besoins exprimés).
2. **Déterminer les acteurs** (utilisateurs ou systèmes externes).
3. **Construire le diagramme** en liant acteurs et cas d'utilisation.
4. **Décrire chaque cas d'utilisation** de manière littéraire (étapes, scénarios, règles de gestion).

3.2.2. Le diagramme d'activités informatique : modéliser le fonctionnement des cas d'utilisation

Concepts clés

Le **diagramme d'activités informatique** permet de détailler **comment le système fonctionne** pour rendre un service, en modélisant les **actions** réalisées par l'application en réponse aux interactions avec les acteurs. Il se concentre sur :

- **Les actions** : unités de traitement réalisées par l'application (ex : « vérifier les informations saisies », « enregistrer un client »).
- **Les nœuds de contrôle** : éléments de coordination (points de décision, fourchettes, jointures) pour représenter les **scénarios alternatifs ou exceptionnels**.
- **Les flots de contrôle** : enchaînement des actions et des décisions.
- **Les activités** : regroupement d'actions liées (ex : « Enregistrer un client »).

Différence avec le diagramme d'activités métier :

- Le diagramme **métier** modélise l'organisation des tâches **manuelles ou informatisées** dans le domaine étudié.
- Le diagramme **informatique** se concentre sur les **traitements automatisés** réalisés par le système pour répondre à un cas d'utilisation.

Notation UML (éléments clés)

1. **Nœud initial** : cercle plein, placé avant la première action.
2. **Action** : rectangle aux coins arrondis, libellé avec un verbe à l'infinitif (ex : « valider la saisie »).
   À noter : Une action représente un traitement déclenché par une interaction de l'utilisateur (ex : validation d'un écran).
3. **Flot de contrôle** : flèche orientée, indiquant le passage d'une action à une autre ou à un nœud de contrôle.
   À noter : On peut ajouter l'événement déclencheur (ex : « validation de l'écran »).
4. **Point de décision** : losange vide, avec des conditions entre crochets pour chaque branche (ex : « [données valides] » ou « [erreur de saisie] »).
5. **Nœud final** : cercle plein entouré d'un cercle, placé après la dernière action.
6. **Fin de flot** : cercle barré d'une croix, pour représenter un arrêt prématuré (ex : abandon du processus).
7. **Action d'appel de comportement** : rectangle avec un **râteau** en bas à droite, pour faire référence à un autre cas d'utilisation ou scénario (ex : appel à « Valider une commande »).

Démarche pour construire le diagramme

1. **Partir du cas d'utilisation** :
   - Identifier les **actions de l'acteur** qui déclenchent une réaction du système (ex : « activer l'enregistrement d'un client », « choisir une commande », « valider la saisie »).
   - Se concentrer uniquement sur les actions **fonctionnelles** (ex : validation, saisie) et ignorer les éléments de navigation (boutons de retour, boîtes de dialogue d'anomalie).

2. **Lister les actions du système** :
   - Pour chaque interaction de l'acteur, détailler les **traitements réalisés par l'application** (ex : « vérifier la cohérence des données », « enregistrer le client », « afficher le récapitulatif »).

- Exemples d’actions système :
    - « Afficher un écran » (ex : écran de saisie).
    - « Contrôler la saisie » (ex : vérifier le format du code postal).
    - « Mettre à jour les données » (ex : enregistrer un nouveau client).

3. **Couvrir tous les scénarios** :

    - **Scénario principal** : enchaînement des actions dans le cas normal.
    - **Scénarios alternatifs** : variantes utiles (ex : client existant vs. nouveau client).
    - **Scénarios exceptionnels** : cas ne permettant pas d’aboutir au résultat (ex : erreur de saisie, données manquantes).
    - Utiliser des **points de décision** pour représenter les choix et les flots de contrôle pour les enchaînements.

Exemple concret

Cas d’utilisation : « Créer un client ».
Fonctionnement détaillé :

1. L’acteur **active l’enregistrement d’un client** → l’application affiche un **écran de saisie** (champs : nom, prénom, adresse, téléphone).
   Action système : « Afficher écran de saisie ».

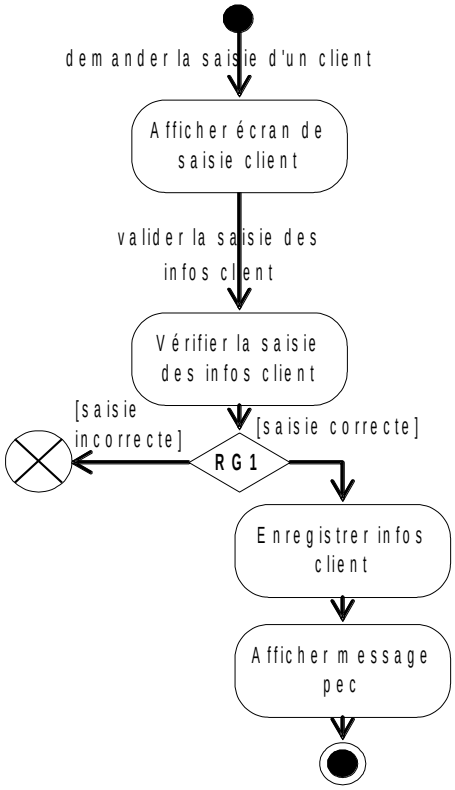
2. L’acteur **valide** les informations saisies → l’application :

    - Vérifie la cohérence des données (ex : format du code postal).
    - Enregistre le client dans le système.
    - Affiche un **écran de confirmation**.
      Actions système :
    - « Vérifier la cohérence des données ».
    - « Enregistrer le client ».
    - « Afficher le récapitulatif ».

3. **Scénarios** :

    - **Principal** : toutes les données sont valides → enregistrement réussi.
    - **Alternatif** : client déjà existant → message « Client déjà enregistré ».
    - **Exceptionnel** : erreur de saisie (ex : code postal invalide) → affichage d’un message d’erreur et retour à l’écran de saisie.

Diagramme :



Points clés à retenir :

- Le diagramme d'activités informatique **détaille le fonctionnement fonctionnel** du système, en complément des cas d'utilisation.
- Il permet de **visualiser les scénarios** et de préparer les **cas de test**.
- Il reste **centré sur les aspects métiers**, sans inclure les détails techniques (ex : navigation entre écrans).

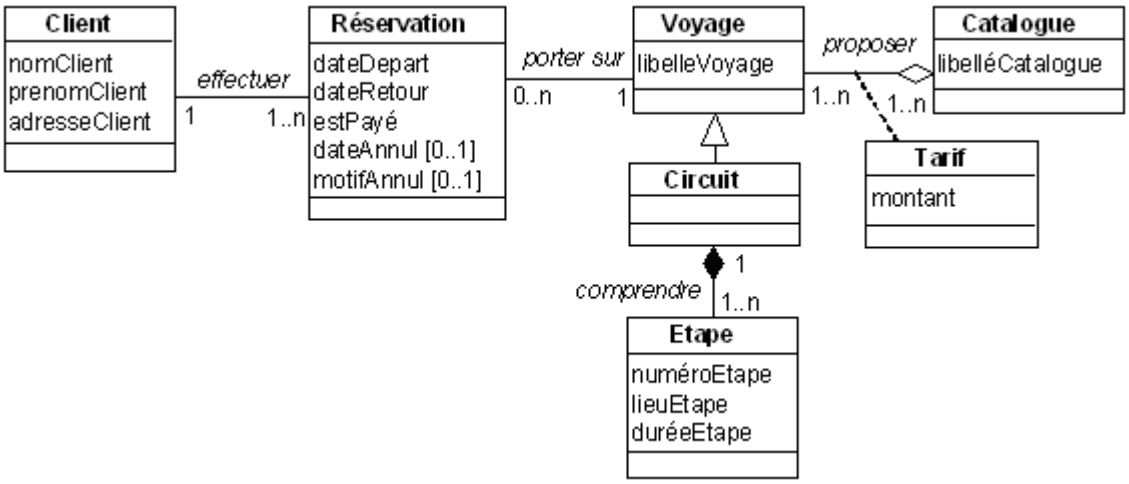
3.2.3. Le diagramme de classes : modéliser la structure des données

Ce diagramme permet de **définir une organisation logique et cohérente des données** en identifiant les **classes essentielles**, leurs **attributs caractéristiques** et leurs **relations fonctionnelles**, afin de modéliser fidèlement la réalité métier.

Concepts clés :

- **Classe** : entité gérée par le système (ex : « Client », « Commande »).
- **Attribut** : caractéristique d'une classe (ex : « nom », « adresse »).
- **Opération** : service offert par une classe (ex : « calculerSolde() »).
- **Relation** :
    - **Association** : lien logique entre classes (ex : « Client → Commande »).
    - **Généralisation** : héritage (ex : « Client » → « ClientParticulier » + « ClientEntreprise »).
    - **Agrégation/Composition** : liens de dépendance forte (ex : une « École » est composée de « Salles de cours »).

Exemple :



Notation UML :

- **Classe** : rectangle divisé en trois compartiments (nom, attributs, opérations).
- **Attribut** : nom en minuscules (ex : « dateNaissance »).
- **Opération** : verbe à l'infinitif (ex : « calculerAge() »).
- **Association** : ligne entre classes, avec une flèche pour indiquer le sens de navigation.
- **Généralisation** : flèche creuse orientée de la classe fille vers la classe mère.

Règles de modélisation :

1. **Ne modéliser que les données nécessaires** aux cas d'utilisation.
2. **Éviter les attributs redondants** (ex : ne pas dupliquer l'adresse du client dans la commande).
3. **Utiliser des classes d'association** pour les informations liées à une relation (ex : « Nombre d'articles commandés »).
4. **Généraliser ou spécialiser** les classes si cela améliore la lisibilité et la réutilisabilité.

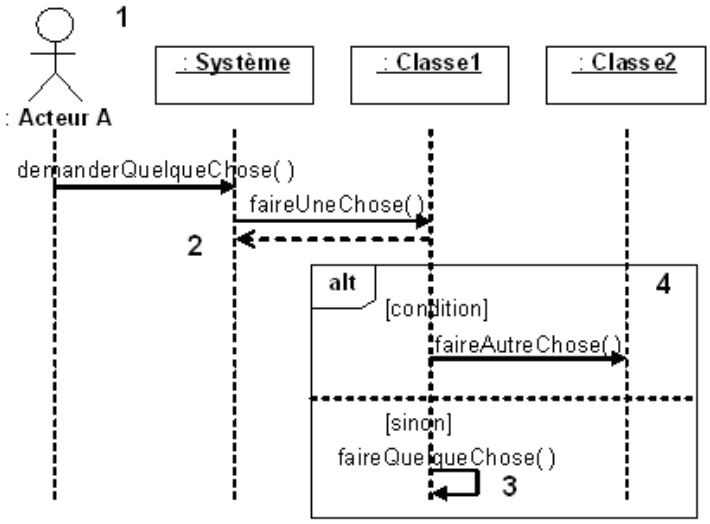
3.2.4. Le diagramme de séquence : décrire le comportement interne

Ce diagramme vise à **représenter dynamiquement les échanges de messages** entre objets collaborant pour accomplir une opération métier, en illustrant **l'ordre chronologique des appels**, les **réponses attendues** et les **interactions internes** du système.

Concepts clés :

- **Objet** : instance de classe (ex : « :Client », « :Commande »).
- **Message** : échange entre objets pour déclencher une opération.
- **Fragment** : sous-ensemble d'interactions (boucle, alternative, option).

Exemple :



Notation UML :

- **Objet** : rectangle vide (ex : « :Client »).
- **Message** : flèche orientée avec un libellé (ex : « calculerTotal() »).
- **Message réflexif** : flèche courbe partant d'un objet et revenant vers lui.
- **Fragment** : rectangle englobant les interactions concernées, avec une étiquette (ex : « alt » pour alternative).

Démarche :

1. **Recenser les objets** impliqués dans le cas d'utilisation.
2. **Lister les interactions** entre acteurs et objets, puis entre objets.
3. **Construire le diagramme** en plaçant les objets de gauche à droite et en reliant les messages.
4. **Identifier les opérations** des classes à partir des messages reçus.

3.2.5. Le diagramme d'état-transition : modéliser le cycle de vie d'un objet

Ce diagramme permet de cartographier l'évolution complète d'un objet au fil de son cycle de vie, en définissant ses états stables, les événements externes qui provoquent les changements, ainsi que les actions et conditions associées à chaque transition,

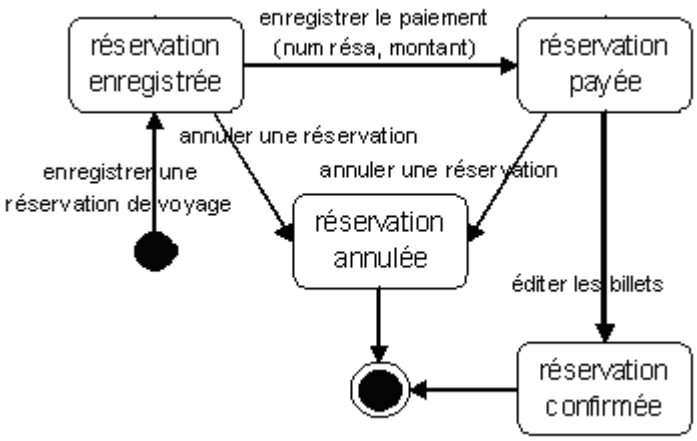
Concepts clés :

- **État** : situation stable d'un objet (ex : « Commande en attente »).
- **Transition** : passage d'un état à un autre (ex : « Commande validée »).
- **Événement déclencheur** : action ou condition provoquant la transition (ex : « Validation par le client »).

Exemple :

Objet **Commande** *:*

- État initial → **Enregistrée** (après « Enregistrer une commande »).
- Transition : [Paiement reçu] → **Payée**.
- Transition : [Annulation] → **Annulée**.
- État final : **confirmée**.



3.3. Synthèse des diagrammes et leur complémentarité

Diagramme	Vue	Utilité en phase d'analyse	Exemple d'utilisation
Cas d'utilisation	Fonctionnelle	Recenser les services et les acteurs.	Identifier que le système doit « Enregistrer une commande ».
Classes	Structurelle	Modéliser les données et leurs relations.	Définir les classes « Client », « Commande », « Produit ».
Séquence	Comportementale	Décrire comment le système rend un service.	Montrer les interactions entre objets pour calculer le total d'une commande.
Activités (informatique)	Comportementale	Détailler le fonctionnement d'un cas d'utilisation.	Modéliser les écrans et les validations pour « Créer un client ».
État-transition	Cycle de vie	Représenter les états d'un objet complexe.	Suivre le cycle de vie d'une « Commande » (enregistrée → payée → livrée).

Pourquoi utiliser ces diagrammes ensemble ?

- **Cohérence** : chaque diagramme apporte un point de vue complémentaire sur le système.
- **Validation** : ils permettent de **croiser les informations** (ex : une classe doit être utilisée dans un cas d'utilisation).
- **Communication** : une **vision globale** facilite les échanges avec la MOA et les utilisateurs.

Bibliographie / sources :

1. Introduction et enjeux de l'analyse orientée objet

- https://dept-info.labri.fr/~beurton/Enseignement/Cours_UML/ResumeCours.pdf
- https://www-inf.telecom-sudparis.eu/cours/CSC4002/EnLigne/Cours/CoursUML/1.html
- https://www.creatis.insa-lyon.fr/~grenier/wp-content/uploads/teaching/MOO/IntroductionOO_full_2pages.pdf
- https://deptinfo.cnam.fr/Enseignement/CycleSpecialisation/MAI/Doc/Intro_MAI13.pdf

2. L'analyse du métier : modéliser l'organisation et les processus

- https://www.iro.umontreal.ca/~dift6803/Transparents/Chapitre2/chapitre2.1.pdf
- https://mcours.net/cours/pdf/info/Introduction_a_la_modelisation_orientee_objets_avec_UML.pdf

3. L'analyse du système informatique : formaliser les services et la structure

- https://www.lacl.fr/~polonowski/Cours/MOO/chapitre1.pdf
- https://perso.liris.cnrs.fr/christine.solnon/coursUML.pdf
- https://www.youtube.com/playlist?list=PLB9AbbTDeBzRskwqyswxrxKI-jmZuoc5y